

توثيق طبقة Management Commands في المشروع

هذه الوثيقة تشرح مجلد `core/management` بالكامل، وبالتحديد الأوامر الموجودة داخل `core/management/commands` ، مع توضيح هدف كل `command`، شكل الـ `arguments`، تدفق التنفيذ داخل الكود، وعلاقته بطبقة الخدمات `core/services` .

المشروع: `django-Patient_data_analysis_system`

تاريخ التوثيق: 23-04-2026

رسالة افتتاحية

في Django، مجلد `management/commands` ليس مجرد مكان لتجميع سكربتات متفرقة، بل هو وسيلة رسمية لتمديد أوامر `manage.py` بأوامر مخصصة للمشروع. هذا يجعل العمليات غير التفاعلية مثل الاستيراد، التصدير، التدريب، التحليل الدوري، أو تنفيذ مهام `batch` جزءًا طبيعيًا من بنية التطبيق بدل أن تكون ملفات خارجية منفصلة. في هذا المشروع، طبقة `management commands` تعمل كواجهة تشغيل نصية لطبقة الذكاء الاصطناعي والتحليل. بمعنى أن الخدمات التحليلية موجودة داخل `core/services` ، لكن أوامر الإدارة تمنحنا طريقة مريحة وآمنة لتشغيلها من الطرفية.

الفكرة العامة لطبقة Management

لماذا هي مهمة؟

- مناسبة للتشغيل اليدوي أو التلقائي.
- تسهل ربط المشروع مع `cron` أو `task scheduler`.
- مفيدة في الاختبار والتجهيز السريع للبيانات.
- تفصل بين منطق التشغيل ومنطق التحليل نفسه.

ما الذي تفعله هذه الطبقة؟

- تشغل وظائف تحليلية من الطرفية.
- تحول الخدمات إلى أوامر `CLI` واضحة.
- تتحقق من صحة `arguments` قبل التنفيذ.
- تعرض نتائج قابلة للقراءة للمشغل أو المطور.

القاعدة المعمارية هنا جميلة: الأوامر نفسها تبقى رقيقة **thin layer**، بينما المنطق الحقيقي يبقى داخل الخدمات. هذا يعني أن `command` ليس هو المكان الذي "يفكر" فيه النظام، بل هو المكان الذي "يشغل" فيه التفكير الموجود أصلاً داخل `service layer`.

نمط الكود المشترك بين جميع الأوامر

جميع الملفات تقريبًا تتبع نفس الهيكل:

```
class Command(BaseCommand):
    """...""" = help

    def add_arguments(self, parser):
        (...)parser.add_argument

    def handle(self, *args, **options):
        ...
        (...)self.stdout.write
```

معنى هذا النمط

- `BaseCommand` : الأساس الذي يوفر سلوك أوامر Django.
 - `help` : النص الذي يظهر عند استعراض الأمر أو طلب المساعدة.
 - `add_arguments()` : تعريف المدخلات التي يقبلها الأمر من CLI.
 - `handle()` : نقطة الدخول الفعلية عند تشغيل الأمر.
 - `CommandError` : طريقة Django الرسمية لإظهار أخطاء منطقية للمستخدم.
- هذا التصميم يجعل كل `command` سهل الفهم: أدخل المعطيات، نفذ منطق الخدمة، ثم اطبع النتيجة.

البنية الحالية لمجلد Management

- `core/management/__init__.py`
- `core/management/commands/__init__.py`
- `core/management/commands/import_diseases.py`
- `core/management/commands/export_ai_dataset.py`
- `core/management/commands/train_risk_model.py`
- `core/management/commands/predict_risk.py`
- `core/management/commands/run_anomaly_detection.py`
- `core/management/commands/run_dbscan_hotspots.py`

شرح الملفات ملفًا ملفًا

Package Marker

1. الملف `core/management/__init__.py`

هذا الملف فارغ، لكنه ضروري ليعامل Python المجلد كحزمة `package`. بدون هذا الملف قد لا تستطيع Django اكتشاف بنية `management commands` بالشكل المتوقع في بعض البيئات أو الأدوات. باختصار: ليس فيه منطق أعمال، لكن له دور بنيوي في تمكين الاكتشاف التلقائي للأوامر.

Package Marker

2. الملف `core/management/commands/__init__.py`

هو أيضًا ملف فارغ، ووظيفته مماثلة: تعريف مجلد `commands` كحزمة Python. Django تبحث عن أوامر مخصصة داخل المسار: `<app>/management/commands` ، ولذلك وجود هذا الملف جزء من البنية القياسية.

Knowledge Ingestion

Data Import

3. الملف `import_diseases.py`

الهدف: استيراد الأمراض المرجعية من ملف CSV مفصول بفاصلة منقوطة ; .

لماذا هذا الأمر مهم؟

لأنه نقطة دخول معرفية للنظام. الأمراض لا تُخزّن فقط كأسماء وأكواد، بل تتحول أثناء الاستيراد إلى كيانات لها `policy_profile` وخصائص مثل `high_priority` و `rare_disease` .

arguments

- `csv-path--` : مطلوب، وهو المسار الكامل لملف CSV.
- `source-name--` : اختياري، ويحدد اسم المصدر المرجعي الذي سيحفظ داخل البيانات.

كيف يعمل الكود؟

1. يبنّي `Path` من قيمة `csv-path` .
2. يتأكد أن الملف موجود وأن امتداده `csv` .
3. يفتح الملف بترميز `utf-8-sig` حتى يتعامل مع BOM إن وجد.
4. يقرأ البيانات عبر `csv.DictReader` مع `delimiter = ;` .
5. يتأكد من وجود الأعمدة المطلوبة.
6. ينظف القيم النصية ويحوّل الحقول الرقمية إلى `int` و `float` .
7. يستدعي `classify_reference_disease()` من `core.services.disease_reference` .
8. ينفذ `update_or_create` على نموذج `Disease` .
9. يطبع في النهاية عدد السجلات التي أنشئت وعدد السجلات التي حُدّثت.

النقطة التقنية الأهم

هذا الأمر لا يستورد البيانات فقط، بل يثريها **enriches data**. أي أنه يدمج طبقة الإدخال مع طبقة التصنيف المعرفي.

مثال تشغيل

```
python manage.py import_diseases --csv-path C:\data\diseases.csv
```

AI Pipeline

Dataset Export

4. الملف `export_ai_dataset.py`

الهدف: تصدير بيانات التشغيل الحالية إلى ملفات CSV جاهزة للاستخدام التحليلي أو التدريب.

arguments

- `output-dir--` : مطلوب، ويحدد المجلد الذي ستنشأ بداخله ملفات CSV.

كيف يعمل الكود؟

1. يبني مسار المجلد الهدف باستخدام `Path(...).expanduser` .
2. يتأكد أن المسار ليس ملفًا عاديًا.
3. يستدعي `export_ai_datasets` () من `core.services.dataset_export` .
4. يطبع عدد الصفوف الناتجة لكل dataset.
5. يطبع المسارات النهائية للملفين الناتجين.

ما الذي ينتجه فعليًا؟

- `visit_level_dataset.csv`

- `disease_region_day_dataset.csv`

هذا الأمر يمثل جسرًا بين قاعدة البيانات التشغيلية وبين طبقة الـ AI أو التحليل الخارجي. هو مفيد جدًا لو أردت لاحقًا تدريب نموذج خارج المشروع أو بناء dashboard في أداة أخرى.

مثال تشغيل

```
python manage.py export_ai_dataset --output-dir .\exports
```

5. الملف `train_risk_model.py`

الهدف: تدريب نموذج baseline لتنبؤ مخاطر الزيارة.

arguments

- `--output-path` : اختياري، ويحدد أين سيتم حفظ ملف JSON الخاص بالنموذج.

تسلسل التنفيذ

1. إذا مرر المستخدم `output-path` يتم تحويله إلى `Path`.
2. يستدعي `train_baseline_risk_model()` من `core.services.risk_prediction`.
3. إذا لم توجد عينات كافية أو `labels` متنوعة بما يكفي، يتم رفع `ValueError`.
4. ال `command` يحول هذا الخطأ إلى `CommandError` مناسب للـ CLI.
5. إذا نجح التدريب، يطبع عدد العينات، ومسار النموذج، وتوزيع `labels`.

ماذا يدير هذا الأمر من بعيد؟

رغم أن الكود في الملف قصير، إلا أنه يشغل `pipeline` كاملة تشمل: `feature engineering`، توليد `min-max labels`، `normalization`، حساب `centroids`، ثم حفظ `JSON artifact`.

مثال تشغيل

```
python manage.py train_risk_model --output-path .\artifacts\risk_model.json
```

6. الملف `predict_risk.py`

الهدف: تنفيذ inference على زيارة معينة باستخدام نموذج المخاطر المدرب.

arguments

- `visit-id--` : مطلوب، وهو الزيارة المستهدفة.
- `model-path--` : اختياري، لتحديد artifact مخصص بدل المسار الافتراضي.

كيف يعمل الكود؟

1. يحاول جلب الزيارة من `Visit.objects.get(id=...)`.
2. إذا لم توجد الزيارة، يرفع `CommandError("Visit not found.")`.
3. يستدعي `predict_visit_risk()`.
4. إذا لم يوجد ملف النموذج أو كان غير صالح، يتحول الخطأ إلى `CommandError`.
5. يطبع في النهاية prediction كاملاً بعد تمريره عبر `serialize_risk_prediction()`.

أهمية هذا الأمر

هذا الأمر هو واجهة inference النصية للنموذج. مفيد في الاختبار السريع، المقارنة بين زيارات، أو ربطه بسكربتات batch مستقبلية.

مثال تشغيل

```
python manage.py predict_risk --visit-id 42 --model-path .\artifacts\risk_model.json
```

Detection

Time-Series Analysis

7. الملف `run_anomaly_detection.py`**الهدف:** تشغيل خوارزمية كشف الشذوذ الزمني من الطرفية.**arguments**

- `disease-id--` : اختياري لتقييد التحليل على مرض واحد.
- `lookback-days--` : عدد الأيام الحديثة التي نريد فحصها، الافتراضي 30.
- `baseline-window-days--` : طول نافذة baseline التاريخية، الافتراضي 7.
- `region-type--` : اختياري لتقييد التحليل على home أو work أو غيره.

منطق التنفيذ

1. يمرر arguments كما هي تقريبًا إلى `detect_temporal_anomalies()`.
2. إذا كانت القيم غير منطقية مثل نافذة أصغر من المطلوب، ترفع الخدمة `ValueError`.
3. الأمر يحول الخطأ إلى `CommandError`.
4. في حالة النجاح، يطبع عدد الشذوذات المكتشفة.
5. ثم يطبع كل candidate بعد تحويله إلى dict بواسطة `serialize_anomaly_candidates()`.

لماذا هذا command مفيد؟

لأنه يحول تحليلًا إحصائيًا مركبًا إلى عملية CLI بسيطة، ويمكن تشغيله يدويًا أو مجدولًا يوميًا أو أسبوعيًا لاكتشاف spikes غير الطبيعية في البيانات.

مثال تشغيل

```
python manage.py run_anomaly_detection --lookback-days 45 --baseline-window-days 10
```

Detection

Spatial Clustering

8. الملف `run_dbscan_hotspots.py`**الهدف:** تشغيل اكتشاف hotspots مكانية بخوارزمية DBSCAN-like من CLI.**arguments**

- `disease-id--` : اختياري.
- `lookback-days--` : النافذة الزمنية الافتراضية إذا لم يعط المستخدم `range` مباشر.
- `date-to--` و `date-from--` : لتحديد فترة تاريخية مباشرة.
- `region-type--` : لتقييد التحليل جغرافيًا حسب نوع المنطقة.
- `eps-km--` : نصف قطر الجوار بالكيلومترات.
- `min-samples--` : أقل عدد نقاط لتكوين `cluster`.
- `persist--` : إذا وُجد هذا `flag`، تحفظ النتائج داخل `GeoCluster`.

المنطق الداخلي

1. يحوّل التواريخ النصية إلى كائنات `Date` بواسطة `parse_date`.
2. يتأكد أن التواريخ بصيغة صحيحة.
3. يشغّل `detect_dbscan_clusters()`.
4. إذا طُلب الحفظ، يشغّل `persist_dbscan_clusters()`.
5. يطبع عدد العناقيد المكتشفة وأرقام العناقيد المحفوظة إن وجدت.
6. يطبع تمثيلًا نصيًا لكل `cluster candidate`.

القيمة التشغيلية

هذا `command` مناسب لتشغيل تحليل مكاني على دفعات، خصوصًا إذا أردنا اكتشاف hotspots دورية بدل انتظار التفعيل التلقائي من `signals`.

مثال تشغيل

```
python manage.py run_dbscan_hotspots --lookback-days 21 --eps-km 2.5 --min-samples 3
--persist
```

العلاقة بين Service Layer و Management Commands

أهم نقطة لفهم هذا المجلد هي أن معظم الملفات فيه لا تحتوي خوارزميات كبيرة. هي فقط orchestrators صغيرة تنقل مدخلات الـ CLI إلى خدمات المشروع. العلاقة كالتالي:

- `import_diseases.py` يعتمد على `disease_reference.py`.
- `export_ai_dataset.py` يعتمد على `dataset_export.py`.
- `train_risk_model.py` و `predict_risk.py` يعتمدان على `risk_prediction.py`.
- `run_anomaly_detection.py` يعتمد على `anomaly_detection.py`.
- `run_dbscan_hotspots.py` يعتمد على `dbscan_hotspots.py`.

هذا يعني أن `command` هنا هو **واجهة تشغيل** وليس **مكان المنطق**. وهذه نقطة تصميم ممتازة لأنها تمنع تكرار الكود وتسمح بإعادة استخدام نفس الخدمة من API أو tests أو signals أو `commands`.

قوة تصميم هذه الطبقة

- الأوامر قصيرة وواضحة، فلا تتضخم بشكل يصعب صيانه.
- كل `command` يتحقق من صحة `input` قبل التنفيذ.
- الأخطاء تتحول إلى `CommandError` بشكل مناسب للمستخدم.
- الطبقة قابلة للربط بسهولة مع `automation` خارجية.
- الأوامر تغطي lifecycle جيدًا: إدخال معرفة، إخراج بيانات، تدريب، inference، وتحليل batch.

ما الذي يمكن تطويره لاحقًا؟

- إضافة أوامر `scheduling` أو `health checks`.
- إضافة تنسيق إخراج JSON رسمي لبعض الأوامر بدل الطباعة النصية فقط.
- إضافة `progress reporting` للعمليات الكبيرة.
- إضافة `flags` للتحكم في `verbosity` بشكل أدق.
- إضافة أوامر لإعادة بناء `reports` أو `clusters` من بيانات تاريخية كاملة.

الخاتمة

إذا كانت طبقة `services` هي العقل التحليلي للمشروع، فإن طبقة `management commands` هي لوحة التشغيل النصية لهذا العقل. هي تسمح للمطور أو المشغل أن يفعل الاستيراد، التصدير، التدريب، التنبؤ، وكشف الأنماط من خارج الـ API بسهولة وبشكل موحد.

ومن الناحية المعمارية، هذه الطبقة ناجحة لأنها تحافظ على مبدأ مهم جدًا: **الـ command يجب أن يبقى رقيقًا، والخدمة يجب أن تبقى ذكية.**

تم إعداد هذه الوثيقة بناءً على الملفات الحالية الموجودة في `src/core/management` و `src/core/management/commands`.